

RESEARCH

Open Access



BPFun: a deep learning framework for bioactive peptide function prediction using multi-label strategy by transformer-driven and sequence rich intrinsic information

Lun Zhu¹, Hao Sun¹ and Sen Yang^{1,2*}

*Correspondence:

Sen Yang

ys@cczu.edu.cn

¹School of Computer Science and Artificial Intelligence Aliyun School of Big Data School of Software, Changzhou University, Changzhou 213164, China

²The Affiliated Changzhou No. 2 People's Hospital of Nanjing Medical University, Changzhou 213164, China

Abstract

Bioactive peptides are beneficial or have physiological effects on the life activities of biological organisms. The functions of bioactive peptides are diverse, usually with one or more, so accurately detecting the multiple functions of multi-functional peptides is extremely important. Traditional experimental identification methods are time-consuming, laborious and costly. To overcome these problems, we adopt a computational biology approach and propose a new model BPFun based on deep learning, which can predict seven functions including anticancer, antibacterial, antihypertensive and so on. In BPFun, we obtained the features of bioactive peptides from different aspects, including biological and physicochemical features. Meanwhile, adopting data augmentation to solve the problem of data imbalance. We combine convolutional networks of different scales and Bi-LSTM layers to obtain high-level feature vectors of different features. Finally, the prediction performance is improved by combining these fused features and combining the self-attention mechanism and the Bi-LSTM layer. Our experiments show that BPFun based on five types of sequence features significantly improves the prediction performance of bioactive peptides. Experiments on the test dataset showed that BPFun gets the accuracy and absolute truth value of 0.6577 and 0.6573 on the dataset of seven functional classifications and was superior to other methods. Codes and data are available at <https://github.com/291357657/BPFun>.

Keywords Multi-label learning, Bioactive peptides, Convolutional neural network, Transformer, Recurrent neural network

Introduction

Bioactive peptides are usually small molecular peptides [1], which are a class of peptides with multiple biological functions and widely exist in animals, plants and foods [2]. As bioactive peptides play an important role in regulating various functions of the body, peptide drugs also have the characteristics of significant curative effect, short half-life and low toxicity [3]. In recent years, more and more bioactive peptides with different functions have



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

been found [4–6]. At the same time, more and more experimental data show that many bioactive peptides have multiple functions. For example, antimicrobial peptides (AMPs) can protect the host from pathogen invasion [7], while some AMPs also have other functions, such as antibacterial, antifungal, antiviral and anticancer [8]. Antiangiogenic peptides (AAPs) are used in the field of cancer treatment because of their function of reducing or inhibiting angiogenesis [5, 9], and they also play an important role in the treatment of other angiogenesis-dependent diseases [10–12]. Some antioxidant peptides (AOPs) extracted from natural proteins may contain additional bioactive properties, such as cholesterol-lowering or antimicrobial properties, in addition to their own antioxidant functions [3, 13]. Accurately predicting the functional activity of bioactive peptides has various meanings: (1) it helps to study the mechanism of action of existing bioactive peptides; (2) it helps to discover new bioactive peptides and to search for bioactive peptides with specific functions; and (3) it helps to develop new peptide drugs to meet the needs of health and safety.

Because bioactive peptides have a wide range of application prospects, it is particularly important to realize the method of large-scale identification of bioactive peptides. Traditional experimental methods cannot meet the needs of peptide function identification, and the accurate identification of bioactive peptides by computational methods has become more and more important. There are quite a few existing methods that can be used to identify bioactive peptides. In 2023, Lee et al. [14] proposed antibacterial peptide function prediction based on the Bert model, named AMP-BERT, which has a bidirectional encoder representation (BERT) architecture from the transformer [15]. In 2022, Pang et al. [16] proposed a method combining transformer neural network architecture and natural language processing knowledge to identify AMP and AMP's functional activity. In 2023, Xu et al. [17] proposed a framework of iAMPCN based on deep learning. The model is mainly based on four types of sequence features and can predict AMP and its functional activity. In 2021, Chen et al. [18] proposed to improve the prediction ability of anti-cancer peptides (ACPs) by using a data augmentation method, called ACP-DA, which could improve the prediction performance of the model while solving the problem of insufficient sample size. In 2022, Bhattarai et al. [19] improved the features used based on Chen [18] to further improve the accuracy of predicting ACPs. In 2019, Zahiri et al. [20] proposed AntAngioCOOL to predict AAPs. They extracted useful description information from different aspects such as PseAAC, atomic spectra and physicochemical spectra to build models. Additionally, another methods [21–24] are usually only applicable to the functional identification of a single bioactive peptide, and only two methods can perform multi-functional identification of bioactive peptides related to the human. One is a deep learning method called MLBP proposed by Tang et al. [25] in 2022 based on convolutional neural networks (CNN) [26] and gated recurrent units (GRU) [27], and the other is a multi-activity prediction method, called MPMABP, for bioactive peptides based on CNN and Bi-LSTM [28] proposed by Li et al. [29] in 2022. Compared to MLBP, MPMABP has been improved by constructing a residual network to ensure that the information of peptides is not lost and replacing GRU with LSTM. However, MPMABP used the same dataset as MLBP, where the variety of bioactive peptides did not increase and did not solve the problem of data imbalance.

The identification of bioactive peptides with multiple functions is a multi-label classification problem, so multi-label classification methods from text, image, and

bioinformatics fields over the past few decades can also be used to identify bioactive peptides [30–36]. In 2008, a multi-label learning method, named CLR, proposed by Johannes et al. [33] introduced an artificial calibration label, which is a method that combines paired preference learning with traditional correlation classification techniques. In 2018, a novel cost-sensitive multi-label learning model, named CPNL, with positive and negative label pair correlation proposed by Wu et al. [31] solves the problem that binary correlation (BR) ignores label correlation. In 2020, another method combining ranking support vector machine and binary correlation with robust low-rank learning, named RBRL, proposed by Wu et al. [34] inherits the ranking loss minimization advantage of ranking support vector machine (Rank-SVM) and overcomes the problem of BR class imbalance and ignoring the problem of label correlation. In 2019, the multi-label deep forest method, named MLDF, proposed by Zhou et al. [35] includes two important parts: measuring perceptual feature reuse and measuring perceptual layer growth. In 2007, Grigorios et al. [36] proposed a multi-label classification method called RAKEL that uses a single-label classifier to consider label correlation.

However, among the multiple methods mentioned above, the multi-label classification method that can be applied to multiple tasks has limited predictive power when used alone for the task of multi-functional identification of bioactive peptides, and the accuracy of identification is also affected by zero-padding of peptide sequences, which cannot provide comprehensive and accurate functional identification. MLBP and MPMABP use peptide sequences as the only input information, and their accuracy decreases significantly when faced with the identification of more categories of peptide functions.

Given these limitations, we propose a new method BPFun for identifying bioactive peptides. For feature construction, we extracted features from multiple perspectives to capture the relationship between peptide sequence information and function. We selected five different feature representation methods as inputs: peptide sequence, peptide sequence encoded by one-hot, peptide sequence encoded by AAindex1 [37], and peptide sequence features obtained using pre-trained models ProtT5 [38] and ESM-2 [39]. The construction process of the BPFun model can be summarized as follows: (1) Obtain and organize a dataset of bioactive peptides and preprocess it, which includes seven bioactive peptides: AMP, ACP, ADP, AHP, AIP, AAP, and AOP; (2) Using AAindex1 and pre-trained models ProtT5 and ESM-2 to process peptide sequences to obtain new encoding and features; (3) Replace the amino acid letters in the peptide sequence with natural numbers to obtain the preprocessed peptide sequence and use one-hot encoding; (4) Using deep learning models to construct BPFun models; (5) Evaluate the performance of the model. Use nine metrics to evaluate the performance of BPFun. Finally, the performance evaluation structure indicates that under the combined action of these five different features, we have effectively improved the representation of peptide sequences. Compared with existing prediction tools, our BPFun has excellent performance in identifying multi-functional peptides and outperforms MLBP and MPMABP in the results of seven functional classifications.

Data and methods

Data and data augmentation

Data

Based on the principle of being able to identify more bioactive peptides, we used nine bioactive peptides related to humans retrieved from the 2020 Google Scholar engine provided by Tang et al. [25], and selected seven of them, including AMP, ACP, ADP, AHP, AIP, AAP, and AOP, with a sample size of more than 100. Among these seven bioactive peptides, the five bioactive peptides AMP, ACP, ADP, AHP, and AIP used the same experimental dataset as MLBP [25] and MPMABP [29], the AAP dataset used the same experimental dataset as AntAngioCOOL [20], and the AOP used the same dataset as AnOxPePred [40]. These datasets were screened using the clustering tool CD-HIT [41] with a threshold set to 0.9. Because there are a large number of sequences with 2–4 amino acids in the AOP dataset, we uniformly processed all data and selected peptide sequences with at least 5 amino acids. We finally obtained 2,409 AMPs, 646 ACPs, 514 ADPs, 868 AHPs, 1,678 AIPs, 134 AAPs, and 318 AOPs. We randomly selected 20% of the dataset as the test dataset and used the remaining 80% as the training dataset. The details of the training and test datasets are shown in Table 1. Apart from single functional bioactive peptides, there are a total of 154 bioactive peptides with two types of functional activity, and there are no bioactive peptides with more functions. Among them, 92 bioactive peptides belong to AMP and ACP; 1 bioactive peptide belongs to ACP and AHP; 2 bioactive peptides belong to ACP and AOP; 4 bioactive peptides belong to ACP and AAP; 22 bioactive peptides belong to ADP and AIP; 14 bioactive peptides belong to ADP and AHP; 3 bioactive peptides belong to AMP and AOP; 2 bioactive peptides belong to ADP and AOP; and 14 bioactive peptides belong to AOP and AHP.

Data augmentation

Due to the imbalance in the quantity of data among different categories, we have adopted a data augmentation approach to increase categories with insufficient data volume. Its basic principle is similar to masking technology, mainly to enhance the range of information in the sequence, and to enhance the generalization ability of the model by simulating information loss. When amino acids are masked, the masked value is uncertain, which increases the range of representation information of the sequence, while only replacing one amino acid to maintain the locality of the sequence without being destroyed. We use a specific character to randomly replace an amino acid in the sequence, and the processed sequence is considered a new sequence, belonging to the same category as the original sequence. It is worth noting that our data augmentation operation is only performed on the training dataset, and no changes will be made to

Table 1 The number of different bioactive peptides in the datasets

Type	Train	Data augmentation	Test	Total
AMP	1927	2000	482	2409 (2482)
ACP	516	1840	130	646 (1970)
ADP	411	1811	103	514 (1914)
AHP	694	1870	174	868 (2044)
AIP	1342	1975	336	1678 (2311)
AAP	107	1289	27	134 (1316)
AOP	254	1383	64	318 (1446)

the test dataset. The added data is placed after the original data by category. Finally, we obtained the results shown in Table 1, excluding duplicate sequences.

Methods

Feature

In the experiment, we used five different types of encoding methods to represent peptide sequences. To ensure the efficiency of our process of obtaining these five sequence representations, we analyzed the length distribution of all data. In the results of length distribution, 99.3% of the sequence length was less than 128, 92.1% of the sequence length was less than 64, and 79% of the sequence length was less than 32. Therefore, we selected the peptide sequence length range from 5 to 32. Sequences with a length less than 5 will be deleted, sequences with a length greater than 32 will be truncated from the first 32 amino acids, and sequences with a length less than 32 will be filled using the zero-padding method, to ensure that there is sufficient sequence information without filling in too many zeros, which will affect data quality. Finally, the data as shown in Table 1 was obtained.

Sequencing information Peptide sequences need to be processed as inputs to the model data. Amino acids {A, C, D, E, F, G, H, I, M, N, P, Q, R, S, T, V, W, Y} will be sequentially transformed into natural numbers {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}, such as sequence {YSKSLPLSVLNP}, will be transformed into {20, 16, 9, 16, 10, 13, 10, 16, 18, 10, 12, 13, 0}. We use the special character 'X' in data augmentation, which will be converted to '0'. In this sequence, its own information will be converted into corresponding numbers, and parts with less than 32 amino acid lengths will be filled with zeros. The sequence is enhanced as follows:

$$S_{original} \xrightarrow{augment} S_{new} \left[\overbrace{s_1, s_2, \dots, X}^{i-1}, \overbrace{s_{i+1}, \dots, s_{32}}^{32-i} \right] \quad (1)$$

where S represents the sequence of bioactive peptides, s_i represents amino acids, and X is a special character.

One-hot encoding One-hot encoding can visually represent amino acid sequences by converting each amino acid into a vector consisting of 0 and 1. The composition of each amino acid is unique. We utilized the result of amino acid transformation to natural numbers, and one-hot encoded the peptide sequence, obtaining a digital matrix with a dimension of 32×21 , where 32 represents the length of the sequence, and 21-dimensional binary vectors represent each amino acid category (including padding zeros). For example, the one-hot encoding of A is {0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}, while the one-hot encoding of C is {0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}. However, one-hot encoding ignores the sequence information between amino acids, that is, it does not take into account the relative positional relationship between amino acids. Therefore, we need to use other feature information together.

AAindex1 AAindex [37] is a database representing numerical indices of various physicochemical and biochemical properties of amino acids and pairs of amino acids. The

AAindex1 database contains statistical indices of the physicochemical properties of 544 amino acids, covering aspects such as hydrophilicity, molecular weight, and molecular volume. We can use AAindex1 encoding to integrate these rich physicochemical property information into the amino acid sequence. We replace the amino acids of a peptide sequence and finally obtain a 32×544 numerical matrix. Unlike one-hot encoding, AAindex1 encoding can more accurately reflect the physicochemical properties and biological features of the amino acid sequence.

ProtT5 The ProtT5 [38] is a model that trains a large number of protein sequences in a self-supervised manner and is only applicable to amino acids with uppercase letters. The features extracted from this self-supervised model (LM embedding) capture important biophysical properties that control protein shape. The structure is shown in Fig. 1. Each amino acid in the amino acid sequence is mapped to a fixed length vector representation through an embedding layer, and position encoding is introduced to encode the relative position of each amino acid in the sequence. Then, the stacking of Transformer encoders is used to capture the relationships in the sequence and generate higher-level representations, thereby obtaining feature information. ProtT5 not only focuses on the sequence itself, but also on the relationship between the sequence and its structure. Therefore, we chose the ProtT5-XL-U50 pre-trained model to extract features of peptide sequences. Finally, we transformed a peptide sequence into a numerical matrix with a dimension of 32×1024 .

ESM-2 ESM-2 [39] is a language model based on Transformer that uses attention mechanisms to learn the interaction patterns between paired amino acids in input sequences. It is also the most high-performance language model developed to date for proteins. Compared to the previous ESM-1b, it not only improves the training parameters and model structure but also increases computational data and resources. On the benchmark of structural prediction, ESM-2 also surpasses other protein language models. Similar to the ProtT5 model, the embedding layer of ESM-2 converts the input amino acid sequence into a high-dimensional vector representation, including words, sentence pairs, and word position information, and optimizes it for protein sequences. By stacking multiple Transformer blocks to capture long-term dependencies in the sequence, a deep representation of the sequence is

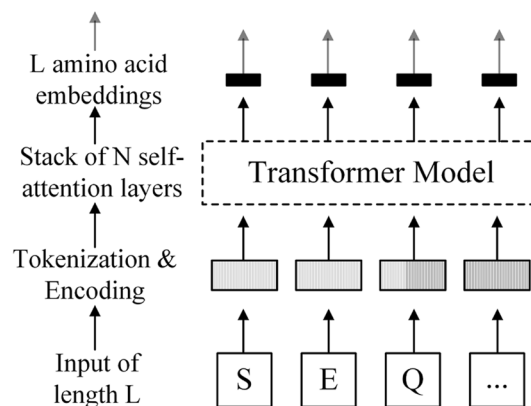


Fig. 1 The general process of feature extraction

generated, and a numerical matrix of 32×1280 is obtained to provide high-quality feature information for subsequent experiments.

Model construction

The structure of BPFun is shown in Fig. 2. This model uses five independent deep learning parts to learn different peptide codes. The details of the model are given in Algorithm 1. In the beginning, we process the data, divide it into a training dataset and a testing dataset, and expand the data on the training dataset (lines 3–4). The resulting data is then encoded or extracted using five independent parts (lines 6–25). Finally, use the obtained sequence information to train the model (lines 27–34).

```

1: Input: Sequence  $S: s_1, s_2, s_3, \dots, s_n$             $\Diamond n$ : Number of sequences
2: Output:  $y = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7\}$         $\Diamond y_i \in [0, 1]$ 
3:  $X, Y = \text{train\_test\_split}(S)$                         $\Diamond X = \text{train\_dataset}(80\%), Y = \text{test\_dataset}(20\%)$ 
4:  $X = \text{augment}(X)$                                       $\Diamond$  Data augmentation
5: /* both on  $X$  and  $Y$     $\text{Seq}$ : Sequence  $X$  or  $Y$  */
6: for  $i = 1$  to  $N$  do                                      $\Diamond N$ : Number of sequences
7:    $S_{\text{tmp}} = \text{padding}(\text{Seq}_i)$                             $\Diamond$  Convert amino acids to natural numbers
8:    $\text{Seq}_1.\text{append}(S_{\text{tmp}})$ 
9: end for
10: for  $i = 1$  to  $N$  do
11:    $S_{\text{tmp}} = \text{one-hot}(\text{Seq}_{1i})$ 
12:    $\text{Seq}_2.\text{append}(S_{\text{tmp}})$ 
13: end for
14: for  $i = 1$  to  $N$  do
15:    $S_{\text{tmp}} = \text{ProtT5}(\text{Seq}_i)$ 
16:    $\text{Seq}_3.\text{append}(S_{\text{tmp}})$ 
17: end for
18: for  $i = 1$  to  $N$  do
19:    $S_{\text{tmp}} = \text{ESM-2}(\text{Seq}_i)$ 
20:    $\text{Seq}_4.\text{append}(S_{\text{tmp}})$ 
21: end for
22: for  $i = 1$  to  $N$  do
23:    $S_{\text{tmp}} = \text{AAindex1}(\text{Seq}_i)$ 
24:    $\text{Seq}_5.\text{append}(S_{\text{tmp}})$ 
25: end for
26: repeat
27:    $\text{feature}_0 = \text{embedding}(\text{Seq}_1)$ 
28:    $\text{feature}_{\text{tmp}} = \text{transformer\_encoder}(\text{Seq}_1)$     $\Diamond$  including positional_embedding
29:   /* both on  $\text{feature}_{\text{tmp}}, \text{Seq}_2, \text{Seq}_3, \text{Seq}_4$  and  $\text{Seq}_5$  */
30:    $\text{feature}_i = \text{CNN}(\text{Seq})$                             $\Diamond i = 1, 2, 3, 4, 5$ 
31:    $\text{feature}_i = \text{Bi-LSTM}(\text{feature}_i)$ 
32:   concatenate all  $\text{feature}_i$                             $\Diamond i = 0, 1, 2, 3, 4, 5$ 
33:   Calculate advanced features through Multi-heads Self-attention and Bi-LSTM
34:   Calculate the result through a sigmoid function
35: until  $\#epoch = n_{\text{epoch}}$ 

```

Algorithm 1 Pseudocode of BPFun

Our model is roughly divided into four parts: the first part is the input, which is used to input the sequence of bioactive peptides; the second part is the deep learning module, which is mainly used to process and learn five different input information and pass it down; the third part mainly integrates the feature vectors passed from the upper layer and improves the performance of the model through self-attention mechanism; the last part uses sigmoid as the activation function of the classification module to output the results. More details of the model will be given in the following section.

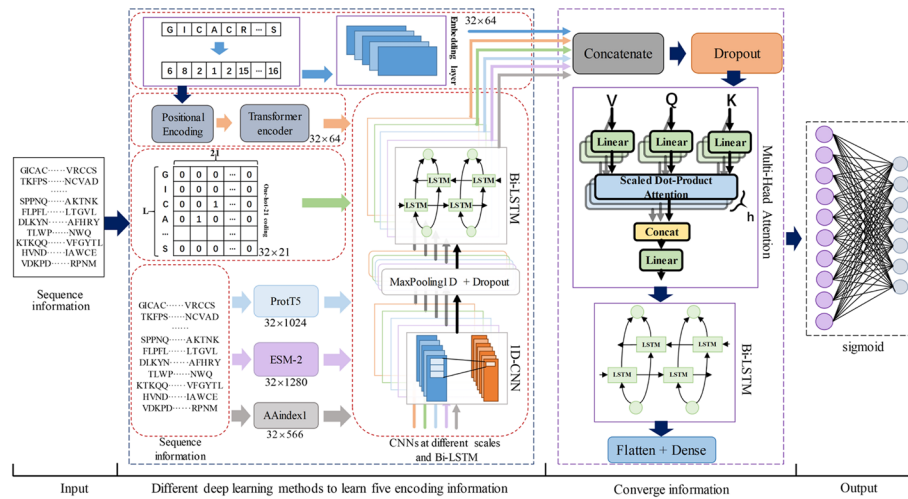


Fig. 2 The structure of BPFun. Using five different sequence representations, feature information of different sequences is extracted from each encoding by combining CNN layers of different scales and Bi-LSTM. Combine the five features through the multi-heads self-attention layer and Bi-LSTM layer, and finally use the sigmoid activation function to obtain the predicted values of different bioactive peptides

Part I The first part is the input part, which is used to provide data. Before inputting the data into the model, we will perform preprocessing, and the specific method has been mentioned in the feature section. After obtaining five different feature information, we will input this feature information into the model. All peptides have a length of 32, and apart from the sequence information that needs further processing, the output shapes of other features are different: one-hot encoded as $(b_s, 32, 21)$; the features extracted by ProtT5 are $(b_s, 32, 1024)$; the features extracted by ESM-2 are $(b_s, 32, 1280)$; AAIindex1 is encoded as $(b_s, 32, 566)$, where b_s represents the size of the batch.

Part II The second part is the deep learning part, where the model will receive feature information transmitted from the input part, including sequence information, one-hot encoding information and so on. In these feature information, sequence information will be processed in two different ways. The first way is through an embedding layer, which converts the positive integer of a vector into a dense continuous feature vector with a fixed size, and the output shape of the embedding layer is $(b_s, 32, 64)$. This usage is the same as processing natural language. However, unlike MLBP [25], in our model, the features processed through embedding layers are not used as the main feature information, and they will form residual connections [42] with other features. The definition of residual connections is as follows:

$$y = F(x) + x \quad (2)$$

where y represents the output after residual connection, $F(x)$ refers to the feature map of the current layer, and x refers to the input of a previous layer. This connection method ensures that the output we need not only includes features of processed peptide sequences but also features of unprocessed peptide sequences, ensuring that information about peptides is not lost during the training process.

The second way is to process it into feature information through the Transformer encoder [43]. The use of Transformer encoders to process peptide sequence information utilizes a multi-layered model structure. The Transformer model includes multiple

encoder and decoder parts, which can be split, making it convenient for us to change the structure of the model as needed. Since the Transformer does not contain a looping part, the security dependency attention mechanism is used to depict global dependencies. Therefore, positional encoding [43] is also required when using the Transformer encoder. The use of absolute positional encoding can integrate positional information into the input of the Transformer, which is achieved by adding the word embedding vector and the word position embedding vector. The formula is as follows:

$$\text{Positional_Encoding}(i) = \text{Embedding_}i + \text{Position_}i \quad (3)$$

where i is a word in the input, its embedding representation is $\text{Embedding_}i$; for the position of i , its embedding is represented as $\text{Position_}i$; "+" represents the addition of vectors element by element, and the dimensions of these two embedded vectors are the same. The output shapes of the position vector and encoder are $(b_s, 32, 64)$, $(b_s, 32, 64)$.

After obtaining the five required feature vectors, the convolutional neural network is used to learn the feature information, where the convolution operation can be represented as:

$$X_j^l = f \left(\sum_{i \in M_j} X_i^{l-1} \otimes K_{ij}^l + b_j^l \right) \quad (4)$$

where X_j^l is the j -th feature map in the l -th layer, X_i^{l-1} is the i -th feature map in the $l-1$ -th layer, K_{ij}^l is the convolution kernel from the i -th feature map in the $l-1$ -th layer to the j -th feature map in the l -th layer, $\otimes$ represents the convolution operation, b_j^l is the bias term of the j -th feature map in the l -th layer, and f is the activation function. We choose to use ReLU as the activation function, which is defined as follows:

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases} \quad (5)$$

To avoid overfitting during the convolution process, we used L2 regularization with the following formula:

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{original}} + \frac{\lambda}{2} \sum_i w_i^2 \quad (6)$$

where $\text{Loss}_{\text{total}}$ is the total loss after L2 regularization, $\text{Loss}_{\text{original}}$ is the original loss, λ is the regularization coefficient used to control the intensity of regularization, w_i is the weight parameter, and $\sum_i w_i^2$ represents the complexity of the weights.

In the experiment, convolutional neural networks were used to extract local features of peptide sequences, and different convolutional kernel sizes can extract different local information. As mentioned in the feature section, the sizes of these five types of feature information are not the same. Therefore, to fully utilize their respective features, we used convolutional neural networks for each input information. When other parameters are the same, the size of the convolutional kernel may not be completely the same. For example, the feature information obtained through the Transformer encoder is suitable for using a convolutional layer of size 2, while the feature information encoded through one-hot is suitable for using a convolutional layer of size 5. Through convolutional layers,

the output shape of features becomes the same, all of which are $(b_s, 32, 64)$, where 64 is the number of filters.

To prevent overfitting issues, we used a max pooling layer and a dropout layer after each convolutional layer to compress the amount of data and parameters. These two layers do not change the shape of the input. Therefore, the output shapes of these two layers are $(b_s, 32, 64)$.

Next is the bidirectional long short-term memory network (Bi-LSTM) layer, which consists of two LSTMs [44] in opposite directions. LSTM is an improved recurrent neural network (RNN) [45] that solves the problems of vanishing and exploding gradients when using recurrent neural networks to process sequences, while also learning long-term dependency information. It includes mechanisms such as forget gates, output gates, and input gates, which are defined as follows:

$$\text{Input gate : } i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (7)$$

$$\text{Output gate : } o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (8)$$

$$\text{Forget gate : } f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (9)$$

$$\text{Cell state : } c_t = (f_t * c_{t-1}) + (i_t * \tanh(W_c[h_{t-1}, x_t] + b_c)) \quad (10)$$

$$\text{Hidden state : } h_t = o_t * \tanh(c_t) \quad (11)$$

where W and b represent the weight matrix and bias term, $[h_{t-1}, x_t]$ represents connecting the hidden state h_{t-1} from the previous moment with the current input x_t into a longer vector, σ is a sigmoid function with an output value between 0 and 1, indicating the degree of information retention. In Bi-LSTM, each LSTM layer uses 50 neurons. Therefore, the output shape of this layer is $(b_s, 32, 100)$.

Part III In the third part, it is necessary to use a concatenate layer to combine the feature information passed down from the previous layers, which includes feature vectors obtained through five Bi-LSTM and sequence feature vectors obtained through an embedding layer. The output shape of this layer is $(b_s, 32, 564)$. Faced with the problem of containing a large number of parameters after combining information, we added a dropout layer after the concatenate layer to reduce the number of parameters in the feature information and alleviate overfitting. The output shape of this layer is $(b_s, 32, 564)$.

Afterward, the output information from the upper layer will enter the multi-head self-attention layer [43], where the main idea of the multi-head self-attention mechanism is to learn multiple attention in different representation spaces to capture different levels and aspects of information. We input the integrated feature information into the multi-head self-attention layer and perform linear transformation through three matrices to obtain three different matrices: query matrix (Q), key matrix (K), and value matrix (V), all of which have the same model dimension. Then, for each head, perform matrix multiplication on Q, K, and V to obtain the score matrix. At the same time, to avoid the gradient disappearing or exploding in backpropagation due to the product value of the matrix being too large or too small, it is necessary to scale the calculated fractional matrix. Specifically, we need to divide the value of each element by the square root of the head dimension d_k , a process commonly referred to as scaling dot product attention

[43]. Multiplying the score matrix by the value matrix yields the corresponding output matrix, as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^t}{\sqrt{d_k}}\right)V \quad (12)$$

where d_k represents the dimension of the word vector. Finally, after calculating each header, we need to merge the results of different headers using the following formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_i)W^o \quad (13)$$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (14)$$

where Q_i, K_i, V_i represents the weight matrices of the linear transformation of the three matrices of the i -th head; W^o is the weight matrix of the linear transformation of the output matrix. The multi-head self-attention layer does not change the shape of the input information, and the output shape of the multi-head self-attention layer is $(b_s, 32, 564)$.

After the multi-head self-attention layer, there is a new Bi-LSTM layer with the same number of neurons as the upper layer, and the output shape of this layer is $(b_s, 32, 100)$.

Next is a Flatten layer, whose main function is to convert the input data into one-dimensional tensors, facilitating the transition of upper-level data to fully connected layers, and helping to solve the problem of multi-dimensional data input. The output shape of this layer is $(b_s, 3200)$.

Afterward, there is a fully connected layer that performs non-linear transformations on the features of the upper layer, extracts the correlations between these features, and maps them to the output space. The output formula is:

$$\text{Output} = \text{ReLU}(\text{dot}(\text{input}, \text{kernel}) + \text{bias}) \quad (15)$$

where input represents the input of the layer, the kernel is the weight matrix of the layer, bias is the bias term. We use ReLU as the activation function and also use L2 regularization. The output shape of this layer is $(b_s, 1024)$, where 1024 is the number of neurons.

Part IV The last part is the output module. In the problem of multi-label classification, each label's classification can be considered independent, so we used the sigmoid activation function. The formula is defined as follows:

$$S(x) = \frac{1}{1 + e^{-x}} \quad (16)$$

The threshold is set to 0.5. If the numerical result corresponding to a certain label in the final output is greater than 0.5, it indicates that the sequence has a function corresponding to the label. In the end, we can obtain all label results that match the functionality of each sequence. For example, if the predicted result of a certain sequence is $\{0.946, 0.550, 0.027, 0.007, 0.010, 0.021, 0.010\}$, then the final label of the sequence will be determined as $\{1, 1, 0, 0, 0, 0, 0\}$, so it can be determined that this peptide sequence belongs to both AMP and ACP, which are bioactive peptides with multiple functions. The output shape of this layer is $(b_s, 7)$.

The loss function used is the mean square deviation, which is used to calculate the mean square of the difference between the predicted value and the true value. The definition of the loss function is as follows:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (17)$$

where n is the number of samples, y_i is the true value of the i -th sample, and $\hat{y}_i$ is the predicted value of the model for the i -th sample.

The optimizer used is Adam, which uses the exponential moving averages r_t and s_t of gradient g_t and the square of gradient g_t . Zero vectors are used to initialize r_t and s_t , and their values update with each iteration t . The following are the Adam parameter updates for each iteration t :

$$g_t = \frac{\partial L}{\partial \omega} \Big|_{\omega_{t-1}} \quad (18)$$

$$r_t = \beta_1 r_{t-1} + (1 - \beta_1) g_t \quad (19)$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2 \quad (20)$$

$$\hat{r}_t = \frac{r_t}{1 - \beta_1^t} \quad (21)$$

$$\hat{s}_t = \frac{s_t}{1 - \beta_2^t} \quad (22)$$

$$\omega_t = \omega_{t-1} - \eta \left(\frac{\hat{r}_t}{\sqrt{\hat{s}_t} + \varepsilon} \right) \quad (23)$$

where ε prevents division by zero, β_1 and β_2 respectively control the decay rates of r_t and s_t , and η ($= 10^{-5}$) represents the learning rate.

The specific optimal parameters in this study are shown in Table 2.

Evaluation metrics

To evaluate the performance of our method, we adopted several evaluation methods, including precision, coverage, accuracy, absolute true value, and absolute false value, which are also the same as the evaluation metrics used by existing models such as MLBP [25] and MPMABP [29]. We also added the F1 value as an evaluation metric. Their definitions are as follows:

$$\text{Precision} = \frac{1}{N} \sum_{i=1}^N \frac{\|L_i \cap L_i^*\|}{\|L_i^*\|} \quad (24)$$

$$\text{Coverage} = \frac{1}{N} \sum_{i=1}^N \frac{\|L_i \cap L_i^*\|}{\|L_i\|} \quad (25)$$

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \frac{\|L_i \cap L_i^*\|}{\|L_i \cup L_i^*\|} \quad (26)$$

Table 2 Optimal configuration values for the model

Hyper-parameters	Optimal values
Activation function	ReLu, sigmoid
Learning rate	1e-5
Embedding dimension	64
Fully connected dimension	1024
Kernel size	2, 5
Pooling size	3
Dropout rate	0.3
Number of heads	8
Optimizer	Adam
Regularization L2	0.001

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Coverage}}{\text{Precision} + \text{Coverage}} \quad (27)$$

$$\text{Absolute true} = \frac{1}{N} \sum_{i=1}^N \Theta(L_i, L_i^*) \quad (28)$$

$$\text{Absolute false} = \frac{1}{N} \sum_{i=1}^N \frac{\|L_i \cup L_i^*\| - \|L_i \cap L_i^*\|}{M} \quad (29)$$

where N is the total number of bioactive peptides, L_i and L_i^* represent the actual and predicted labels for sample i , $\cap$ represents the intersection and $\cup$ represents the union, $\|A\|$ represents the number of elements in A , M represents the function type of the low-redundancy dataset, and Θ is defined as:

$$\Theta(L_i, L_i^*) = \begin{cases} 1, & \text{if } L_i^* \text{ is identical to } L_i \\ 0, & \text{other} \end{cases} \quad (30)$$

We used sensitivity (SEN), specificity (SPE), and Matthews Correlation Coefficient (MCC) [46–49]. For a given class, peptides with the function of that class are considered positive samples, and other classes without the function of that class are considered negative samples. The formula is as follows:

$$\text{SEN} = \frac{TP}{TP + FN} \quad (31)$$

$$\text{SPE} = \frac{TN}{TN + FP} \quad (32)$$

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (33)$$

TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively.

Results

Distribution of amino acids

To analyze the distribution of amino acids in the sequence, we used weblogo [50] to generate sequence logos. The residues appearing at each position in the sequence are drawn graphically, and the cumulative residues at each position can reflect the consistency of

residues at that position. The size of the graphical character corresponding to an amino acid is proportional to the frequency of the residue at that position. Because entropy values are designed during the calculation process, if the sequence letters in a column are more chaotic, the entropy value will be larger, and the letters will be shorter; conversely, if there is more regularity, the entropy value will be smaller, and the letters will be taller. The amino acid graphics for each multi-functional peptide are shown in Fig. 3. Among these seven bioactive peptides, there is no position where a certain amino acid is always present, and most positions have multiple amino acids that can be substituted. In ACP, alanine (A), glycine (G), lysine (K), and leucine (L) are predominant, and the first amino acid usually appears in alanine, glycine, lysine, and phenylalanine (F); in ADP, alanine, leucine, and valine (V) are predominant; in AHP, proline (P) is predominant; in AIP, alanine and leucine are predominant; in AOP, leucine and proline are usually predominant, and the 23-th amino acid in the sequence is usually cysteine or leucine or tyrosine (Y); in AAP, cysteine, glycine, arginine, and serine are predominant; AMP is dominated by glycine and lysine. In general, these seven bioactive peptides do not have a certain amino acid fragment that always appears, and there are one or more amino acids that can be substituted at each position.

Feature comparison

Different lengths of sequences

We chose to set the length of the sequence used in the experiment to 32 based on the fact that 79% of the data had a length less than 32. As shown in Table 3, we trained on different lengths. When the sequence length was 256 residues, the accuracy of these seven bioactive peptides was 0.634, while when there were only 32 residues, the accuracy was 0.649, with a difference of about 2%. The overall trend is towards a slight increase in accuracy as the length of the sequence decreases. The other metrics also reflect this trend. And we also analyzed the time spent processing a piece of data when using different lengths, as shown in Table 4. For the length of a sequence, longer sequences require longer processing time, especially when using the pre-trained model ProtT5 to extract sequence features, which takes a lot of time. In this experiment, reducing the length of the sequence used to a certain extent can almost ignore the impact on the experimental

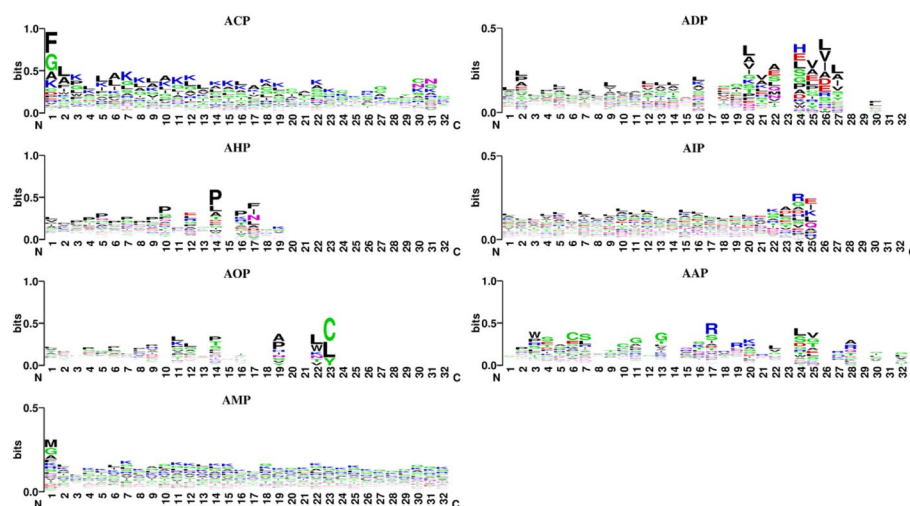


Fig. 3 The distribution of the first 32 amino acids of seven different bioactive peptides

Table 3 The performance of BPFun with different lengths of sequence

Length	Precision	Coverage	Accuracy	F1	Absolute true	Absolute false	Sen	Sp	MCC
32	0.6490	0.6538	0.6490	0.6514	0.6442	0.0858	0.6177	0.9622	0.6229
64	0.6424	0.6428	0.6424	0.6426	0.6420	0.0853	0.6108	0.9626	0.6192
128	0.6347	0.6359	0.6347	0.6353	0.6344	0.0883	0.6057	0.9604	0.6164
256	0.6344	0.6349	0.6344	0.6346	0.6344	0.0844	0.6061	0.9537	0.6100

Table 4 The time consumed by sequences of different lengths

Length	Sequence	One-hot	ProtT5	ESM-2	AAindex1	Total Time
32	0.0135 s	0.0075 s	0.6734 s	0.1535 s	0.0570 s	0.9049 s
64	0.0146 s	0.0123 s	1.0293 s	0.3180 s	0.1114 s	1.4856 s
128	0.0166 s	0.0232 s	2.1645 s	0.5093 s	0.2821 s	2.9957 s
256	0.0210 s	0.0435 s	3.2488 s	0.9418 s	0.5050 s	4.7601 s
517	0.0297 s	0.0889 s	7.2219 s	2.0074 s	1.3619 s	10.7098 s

results, but it can save a lot of time for the entire experimental process. We believe that reducing the sequence length appropriately will not affect the experimental results, but can greatly reduce time consumption during the experiment. Therefore, in this experiment, we selected the first 32 residues of the truncated sequence and filled in a small amount of zeros when insufficient.

Different numbers of sequences

When increasing the number of datasets, considering the different types of bioactive peptides with varying quantities, we chose to start with 500 and increase each time, up to 2500, to test whether different quantities have an impact on the performance of the BPFun model. Firstly, to ensure that the original dataset is used, no processing will be performed when the number of bioactive peptides exceeds the value we have selected. Subsequently, increase the quantity of bioactive peptides that are less than the selected value until the selected quantity is reached. As shown in Table 5, the metrics vary for different quantities of bioactive peptides. In terms of Precision alone, the more bioactive peptides there are between 500 and 2000, the higher the Precision. When the number of bioactive peptides is 2500, Precision will decrease. The other metrics have similar patterns of change. Overall, the performance of the model is best when the number of each bioactive peptide is controlled in 2000.

Classification comparison of different features

In addition to the impact of different lengths on the research, we also analyzed the impact of different encoding methods or feature extraction methods on the features of different bioactive peptides. All sequences have a length of 32, and we encode them using three encoding methods: the number sequence converted from the original sequence, one-hot, and AAindex1; ProtT5 and ESM-2 pre-trained models extract preliminary features. Subsequently, we input these five features into the decision tree classifier. The average data from multiple experiments and the results obtained for the given model are shown in Fig. 4, the feature representations obtained using one-hot encoding and ESM-2 pre-training model can improve the predictive performance of these seven bioactive peptides, with higher ACC values compared to others; AAPs and AOPs have achieved good results in encoding and feature extraction using these five methods. We tested the impact of features obtained through different methods on our model using

Table 5 The performance of BPFun with different numbers of sequences

Number	Precision	Coverage	Accuracy	F1	Absolute true	Absolute false	Sen	Sp	MCC
500	0.5989	0.5990	0.5989	0.5989	0.5987	0.0887	0.6017	0.9638	0.6157
1000	0.6154	0.6168	0.6154	0.6158	0.6140	0.0899	0.6527	0.9573	0.6352
1500	0.6240	0.6251	0.6241	0.6244	0.6230	0.0909	0.6481	0.9573	0.6318
2000	0.6515	0.6530	0.6515	0.6520	0.6500	0.0878	0.6725	0.9527	0.6369
2500	0.6363	0.6375	0.6363	0.6367	0.6350	0.0926	0.6161	0.9613	0.6191



Fig. 4 Heat map visualization of decision tree classifier results based on different features

different combinations, and the results are shown in Table S1. Among them, two features obtained by pre-trained model contribute more to our model, while the other three features play an important role in improving the model performance.

Feature visualization

In the experiment, we combined sequence information, One-hot and AAindex1 encoding information, and feature information extracted using pre-trained models ProtT5 and ESM-2 to represent peptides. To verify that all five types of information contribute to predicting the multiple functions of peptides, we visualized the original dataset using UMAP [51] on both the training and testing datasets. Project the initial encoding vectors and high-level abstract features of all samples extracted from the model based on different input information onto a two-dimensional space.

For each type of peptide information, using the corresponding model for learning will result in better feature representations. As shown in Fig. 5, we visualized the feature information from both the training process and the independent test dataset. During the training process, using five different initial peptide sequence information alone did not effectively aggregate the same category in the results. However, when using these five different types of information simultaneously, the results were able to distinguish different categories well. On the independent test dataset, the identification ability of the five individual peptide sequence information is not up to that of the comprehensive bioactive peptide information. It can be seen that compared to using single information to identify the function of peptides, using different information from bioactive peptides can achieve better predictive ability. Therefore, this model can learn abstract representations of different bioactive peptides from the initial sequence information of peptides,

information encoded using one-hot and AAindex1, and information extracted from pre-trained models ProtT5 and ESM-2, improving the ability to identify peptide functions.

Comparison with other models

To verify the stability of BPFun, we conducted a five-fold cross-validation experiment with other multi-label classification methods. So far, MLBP and MPMABP are the latest methods for classifying multi-functional labeled bioactive peptides. MLBP utilizes a deep learning method that combines CNN and Bi-GRU; MPMABP constructed a residual network based on MLBP and replaced GRU with LSTM. At the same time, we have also chosen

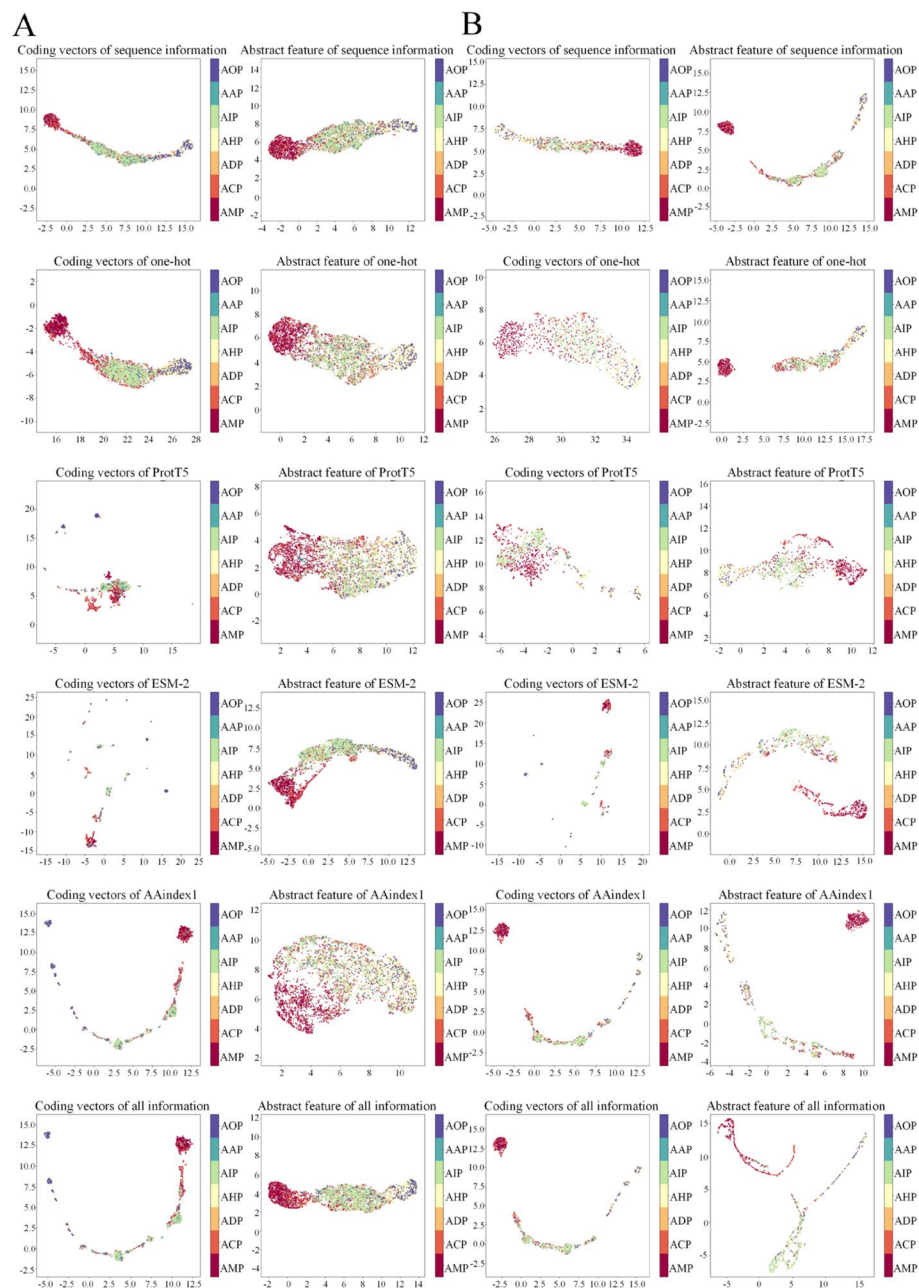


Fig. 5 UMAP visualization results on the dataset: **A** using five features individually and simultaneously on the training dataset; **B** using five features individually and simultaneously on the test dataset

other multi-label algorithms that can be used to predict bioactive peptides, such as sorted support vector machines and binary correlation with robust low-rank learning (RBRL), which inherit the ranking loss minimization advantage of rank support vector machine (Rank-SVM) and overcomes the problem of BR class imbalance and ignoring the problem of label correlation; multi-label learning with deep forest (MLDF), which utilizes measurement perceptual feature reuse and measurement perceptual layer growth; calibrated label sorting (CLR) combines pairwise preference learning with traditional correlation classification techniques; random k-label set (RAKEL), considering label correlation by using a single label classifier. We use the five-fold stratified cross validation on the training dataset. To avoid overfitting during the five-fold cross-validation experiment on the expanded training dataset, which may lead to biased experimental results, we chose to experiment on the original training dataset. As shown in Table 6, BPFun outperforms the above methods in eight aspects: Precision, Coverage, Accuracy, F1 value, Absolute true, Absolute false, Sen, and MCC. Compared with other method, BPFun has improved Precision by 0.022–0.178, Coverage by 0.02–0.179, Accuracy by 0.022–0.178, F1 value by 0.021–0.179, Absolute true by 0.024–0.178, and Absolute false by 0.008–0.036. The deviation of Precision, Coverage, Accuracy, F1 value and Absolute true is maintained at 0.011, and the deviation of Absolute false is maintained at 0.002. The results indicate that our model has an excellent performance in identifying multi-functional peptides, while also exhibiting certain stability.

To verify the performance of the model on an independent test dataset, we used the expanded training dataset for training. In the end, our method outperformed other methods among the eight metrics. As shown in Table 7, our method BPFun still achieved the best results on our dataset containing seven types of multi-functional peptides. Precision increased by 0.0376–0.1378, Coverage increased by 0.0359–0.1187, Accuracy increased by 0.0376–0.1378, F1 value increased by 0.0367–0.1193, Absolute true increased by 0.0391–0.1211, and Absolute false decreased by 0.0032–0.0287. The general multi-label classification methods RBRL, MLDF, CLR, and RAKEL have slight limitations in their ability to specifically predict multiple functions of bioactive peptides. MPMABP performs well in predicting five types of multi-functional peptides but performs poorly in predicting seven types of multi-functional peptides. The fundamental reason is that the model has multiple convolutions of different scales, which cannot effectively extract feature learning when encountering test datasets with small sample sizes. MLBP has good performance in predicting five and seven types of multi-functional peptides, but its prediction accuracy is still insufficient when only peptide sequence information is available. Further confirmation confirms that our method utilizes different feature representations to enhance the information content of peptides, making it more effective in predicting peptide functions. It also demonstrates that this method has higher complexity and stronger learning ability, providing a useful reference for predicting the detection rate of peptides from other species.

Because the ROC and PR curves can more intuitively compare the performance of models, this paper presents the ROC and PR curves of BPFun and six other models on independent test datasets, as shown in Fig. 6. The AP and AUC of BPFun are 0.726 and 0.923, showing better performance compared to other methods. This indicates that our method has good predictive ability in predicting multi-functional peptides.

Table 6 Five-fold cross validation

Model	Precision	Coverage	Accuracy	F1	Absolute true	Absolute false	Sen	Sp	MCC
MLBP	0.636±0.016	0.640±0.017	0.636±0.016	0.638±0.016	0.633±0.016	0.090±0.003	0.634±0.013	0.956±0.002	0.618±0.014
MPMABP	0.587±0.006	0.587±0.006	0.587±0.006	0.587±0.006	0.587±0.006	0.092±0.003	0.588±0.012	0.961±0.002	0.597±0.009
RBRL	0.588±0.013	0.588±0.014	0.588±0.013	0.588±0.013	0.588±0.013	0.118±0.004	0.588±0.014	0.931±0.002	0.519±0.016
MLDF	0.480±0.102	0.481±0.101	0.480±0.102	0.480±0.101	0.479±0.103	0.092±0.008	0.358±0.144	0.976±0.008	0.440±0.157
CLR	0.529±0.016	0.593±0.021	0.529±0.016	0.559±0.018	0.593±0.021	0.107±0.004	0.599±0.047	0.948±0.021	0.593±0.017
RAKEL	0.602±0.009	0.626±0.011	0.602±0.009	0.614±0.010	0.626±0.011	0.096±0.003	0.620±0.052	0.945±0.021	0.521±0.032
BPFun(our)	0.658±0.011	0.660±0.011	0.658±0.011	0.659±0.011	0.657±0.011	0.082±0.002	0.657±0.015	0.966±0.007	0.625±0.013

We conducted prediction results distribution for different multi-label methods and presented the relationships between sample points through UMAP. The horizontal and vertical coordinates do not represent specific biological or mathematical features, but rather represent the relative positional relationships of sample points in low dimensional space. The distance between two sample points represents their differences in the original high-dimensional data. As shown in Fig. 7. Among the six compared methods, only the prediction results of RBRL and MLDF were able to clearly distinguish different categories of bioactive peptides; the other four methods, CLR, RAKEL, MPMABP, and MLBP, can only predict a large number of categories, and in most cases, they cannot effectively distinguish the categories to which they belong. Our method BPFun achieved good results in predicting these seven different bioactive peptides, with clear boundaries between different categories and good clustering of results.

Various bioactive peptides

To verify the predictive ability of the BPFun method for single functional bioactive peptides, we separately plotted ROC and PR curves for seven types of bioactive peptides, as shown in Fig. 8. According to the ROC curve, AMP, AHP, and AIP have higher AUC values of 0.891, 0.940, and 0.926, while the AUC values of the other four bioactive peptides are slightly lower. In the PR curve, AMP, AHP, and AIP also performed well, with higher AP values of 0.837, 0.743, and 0.762. The AP values of the other four bioactive peptides performed poorly.

We also analyzed the MCC values of various methods for each class of bioactive peptides, as shown in Fig. 9. Our method BPFun performs better than the comparison method in terms of MCC values for AMP, ADP, AHP, and AIP, indicating that BPFun has better predictive ability for these four bioactive peptides than the comparison model. Our method also achieved at least moderate performance in the MCC values of ACP, AAP, and AOP, with little difference compared to the better performing methods.

We compared the sensitivity and specificity between the original dataset and the current dataset. As shown in Fig. 10. On the original dataset, the specificity of ADP, AAP, and AOP was 1, while the sensitivity was 0. This indicates that due to the imbalance of data between different categories in the original dataset, the accuracy of prediction is affected. When predicting these three types of functional peptides, the model judges most of the samples as negative samples. We alleviate the error prediction caused by data imbalance by increasing the amount of data for these less-data categories. From the results, it can be seen that the specificity of ADP, AAP, and AOP did not significantly decrease, while the sensitivity was effectively improved; the proportion of other categories in all data has decreased, and there is some fluctuation in sensitivity. From the experimental results, it can be seen that our method of expanding data can to some extent alleviate the problem of data imbalance between multiple categories of data. We alleviate the erroneous predictions caused by data imbalance by increasing the amount of data for these categories with less data. The originally insufficient data has been expanded to expand the representation range of the dataset. Most enhancement techniques involve constructing similar objects, which can result in similar sequences, which is a common problem in data augmentation techniques. However, some datasets are too small, making it difficult for the model to pay attention to the representation range of this type of data. By using enhanced methods to expand the representation range of this

Table 7 Independent test dataset

Model	Precision	Coverage	Accuracy	F1	Abso- lute true	Abso- lute false	Sen	Sp	MCC
MLBP	0.6201	0.6221	0.6201	0.6211	0.6182	0.0855	0.6511	0.9467	0.6053
MPMABP	0.5758	0.5792	0.5758	0.5775	0.5724	0.0917	0.5461	0.9643	0.5741
RBRL	0.6116	0.6116	0.6116	0.6116	0.6116	0.1110	0.0266	0.9884	0.0455
MLDF	0.5377	0.5393	0.5377	0.5385	0.5362	0.0952	0.5636	0.9519	0.5516
CLR	0.5199	0.5853	0.5199	0.5507	0.5853	0.1085	0.8980	0.6707	0.4045
RAKEL	0.5921	0.6157	0.5921	0.6037	0.6157	0.0986	0.4547	0.9622	0.4920
BPFun(our)	0.6577	0.6580	0.6577	0.6578	0.6573	0.0823	0.6725	0.9527	0.6369

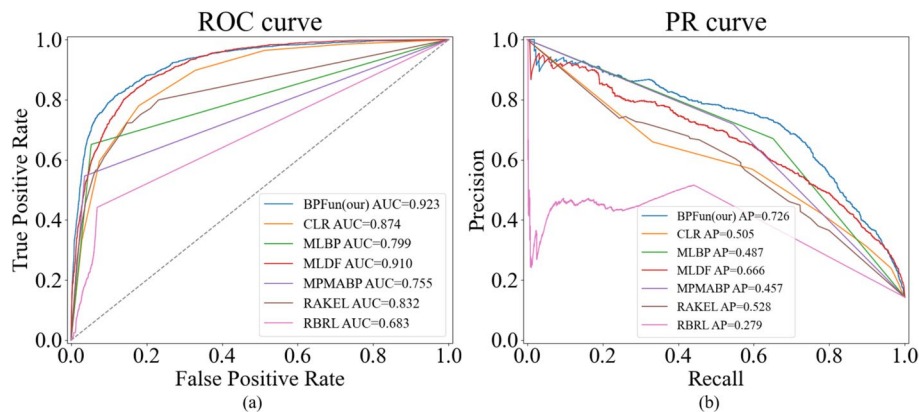


Fig. 6 ROC and PR curves of BPFun and other models: **a** ROC curve; **b** PR curve

type of data, the problem of limited data volume has been improved to some extent. Therefore, although there may be overfitting for learning single category data, this overfitting is within a reasonable range and will not expand to other categories. From the results, it can be seen that the specificity of ADP, AAP, and AOP did not significantly decrease, while the sensitivity was effectively improved; the proportion of other categories in all data has decreased, and there is some fluctuation in sensitivity, but the change is small. From the experimental results, it can be seen that our method of expanding data can to some extent alleviate the problem of data imbalance between multiple categories of data.

Case study

In the above content, we constructed a multi-label bioactive peptide predictor BPFun and compared it with other different methods. The final results showed that our method BPFun outperformed other methods in predicting multi-functional bioactive peptides. To further demonstrate the predictive ability of BPFun, we randomly selected some bioactive peptides for prediction, which included randomly selected data used in experiments and some data that had never been used before. Table 8 presents the predicted results of our method and two other comparative methods for these bioactive peptides.

Our method BPFun can predict the activity of the vast majority of bioactive peptides, but there are shortcomings in predicting antioxidant peptide AOP. On the contrary, the MLBP method performs well in predicting AOP, but its overall performance is not as good as our method in the selected bioactive peptide sequences; The MPMABP method has insufficient ability in predicting the activity of seven bioactive peptides. From the

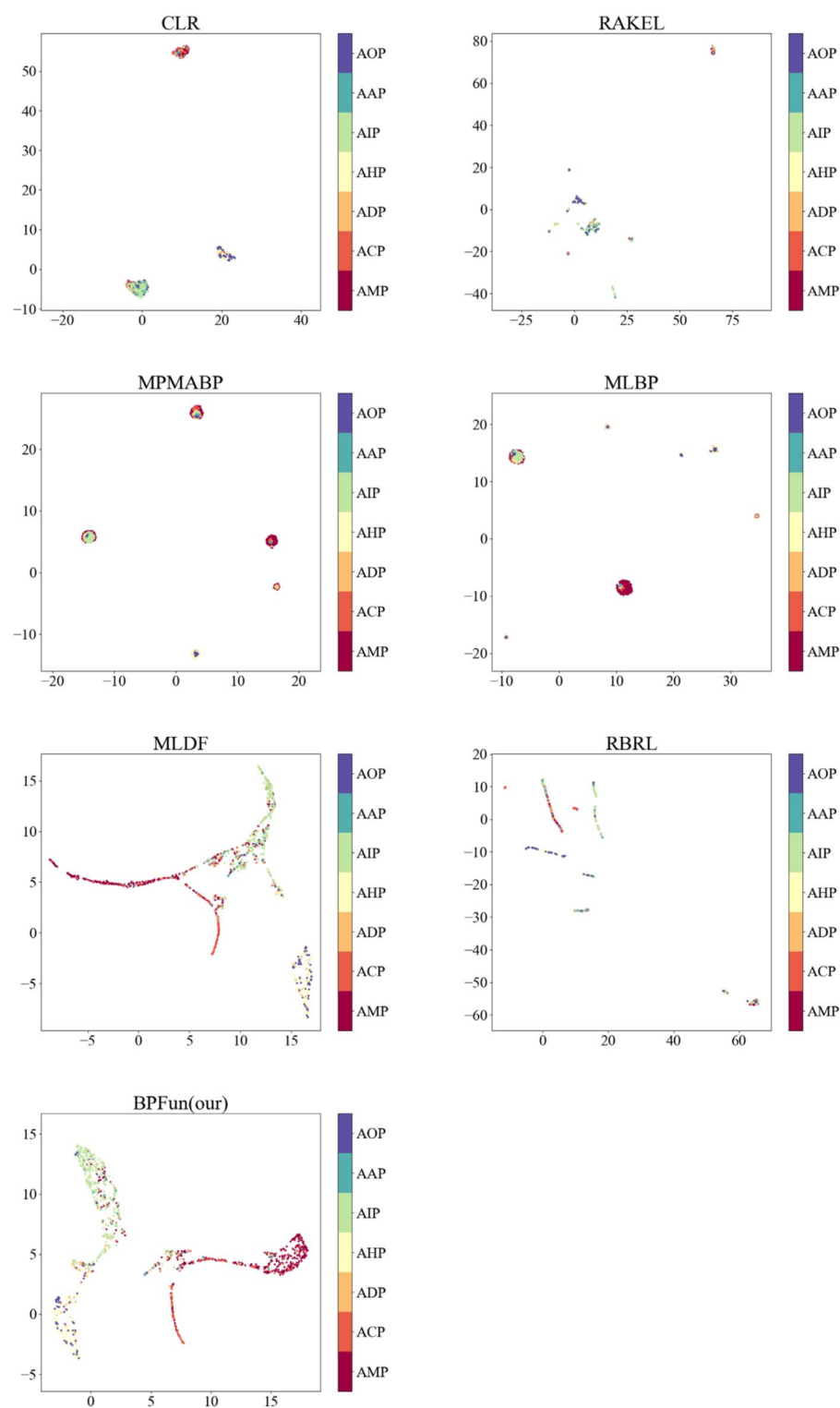


Fig. 7 Prediction results of BPFun and other methods

prediction results of the above examples, our method BPFun is superior to existing methods in predicting multi-functional bioactive peptides.

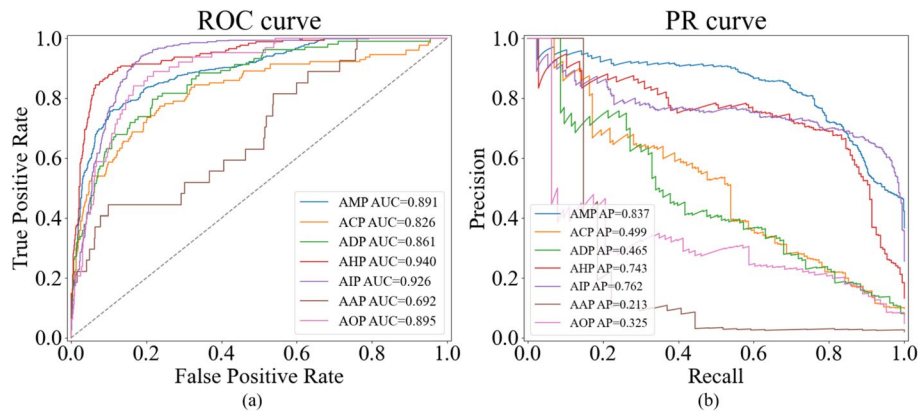


Fig. 8 ROC and PR curves of different types of bioactive peptides: **a** ROC curve; **b** PR curve

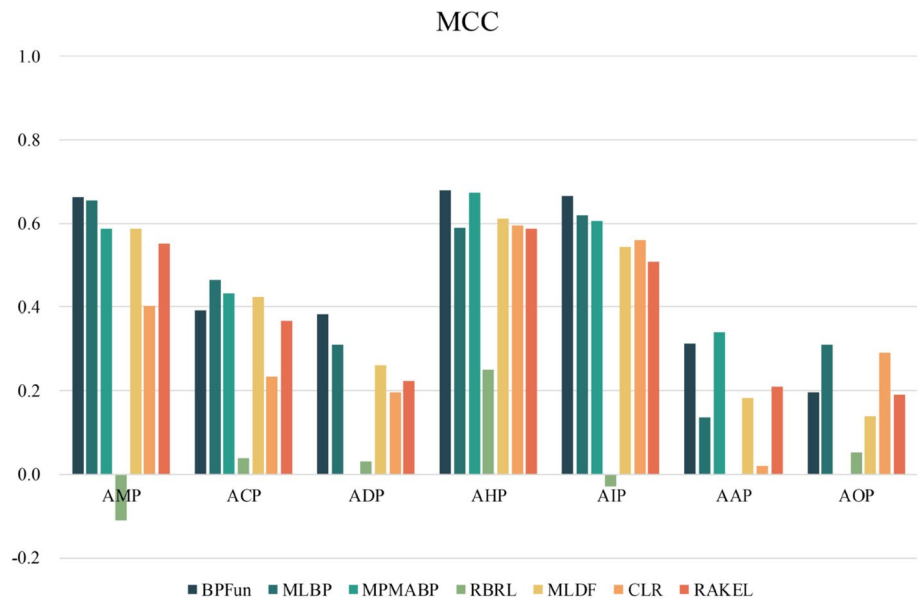


Fig. 9 MCC values of different multi-label methods on the test dataset of per-class bioactive peptides

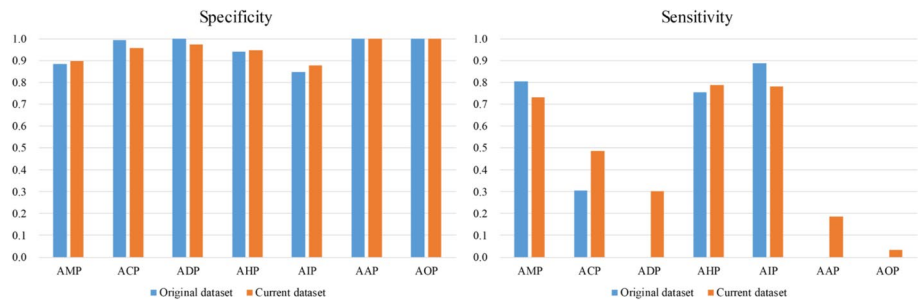


Fig. 10 SEN and SPE of BPFun on the original dataset and the current dataset

Discussion

Due to the scarcity of bioactive peptides that can determine their functional activity, and the irregular composition of amino acids, accurately predicting the functional activity of bioactive peptides remains challenging. A single feature vector is not sufficient to

Table 8 Comparison of BPFun against two other methods with a case study

Sequence	True labels	Prediction		
		BPFun	MLBP	MPMABP
NCNFLCKVKQRLRSVSSTSHIGMAIPRPG	AMP	AMP	AMP	None
FAKKLLAKALKL	ACP	ACP	ACP	ACP
GIPCGESCVFIPCITGAIGCSCKSKVCYRN	ACP	ACP	ACP	ACP
SQWSPCSRTC GGGVSFRER	AAP	AAP	AAP	AAP
GRGKRRREKQRPCDKPRR	AAP	AAP	None	None
CKITRCPMIPCYISSPDECLWMDVWTEKNINGHQAKFFAC	AAP	AAP	AAP	AMP
LNSGDALRVPSGTTY	AOP	AIP	AOP	AIP
AMRLTGNRPCLYAT	AOP	AIP	AOP	None
SALLRSIPA	AOP	ADP	ADP	None
ACYCRIGACVSGERLTGACGLNGRIYRLCCR	AMP/ACP	AMP/ACP	AMP	AMP/ACP
ALKAALLAILKIVRVIKK	AMP/ACP	AMP/ACP	ACP	ACP
GFGALFKFLAKKVAKTVAKQAAKQGAQYVWNQME	AMP/ACP	AMP/ACP	AMP/ACP	AMP/ACP
TGHMILAYMEDHLRNRDR	ADP/AIP	ADP/AIP	ADP	AIP
FALPQYLK	AHP/AOP	AHP	AHP	None

effectively provide the demand for predicting bioactive peptides. Therefore, to improve the performance of the model in predicting bioactive peptides, we used different features to represent the same bioactive peptide, to fully utilize the information of a bioactive peptide sequence. Through visual analysis of features, it can be concluded that using the five features we have selected simultaneously is far more suitable for predicting the function of bioactive peptides than using only one feature alone. Therefore, we propose a prediction method for multi-functional bioactive peptides, called BPFun, which uses multiple feature information to predict bioactive peptides. From our experimental results, it can be seen that this method has good comprehensive performance. Compared with existing methods, BPFun has better performance in predicting bioactive peptides. We believe that the following reasons have improved the performance of BPFun.

First, we use different features to represent peptide sequences. We used five different features, including the original sequence, to describe sequence information from different perspectives. In the experiment, we made heat maps for different categories of bioactive peptides and different feature representations, and the results showed that these five different features can effectively represent the information of peptide sequences. Compared to using a single feature as input, our approach of using different features can effectively improve model performance. Among the existing methods for identifying bioactive peptides, using multiple different features can achieve more significant improvements in experiments compared to using a single feature method [18, 19].

Second, we use data augmentation to address the issue of data imbalance between different categories, especially after the number of bioactive peptide categories has increased to seven. Data augmentation is the process of generating pseudo samples by modifying the original sequence: randomly replacing a certain amino acid letter in the sequence with a specific character, and then obtaining a new sequence. The dataset obtained through data augmentation can train the model more effectively compared to the dataset without data augmentation. The model's predictive ability for categories with smaller sample sizes is significantly improved, and the sensitivity of these categories with smaller sample sizes can be significantly improved.

Finally, we optimized the model. Using convolutional layers of different scales to extract different features can effectively utilize the different features we use. The use of Bi-LSTM can effectively utilize the contextual information of peptide sequences. Meanwhile, the attention mechanism used in the model effectively improves the performance of the model. Under the same dataset conditions, our method has better performance and overall performance compared to other methods.

In summary, our proposed method BPFun has good performance in predicting the function of bioactive peptides. The different optimization methods used can effectively improve the performance of the model. Among them, using multiple different representation methods to describe sequence information and data augmentation of samples can effectively alleviate the problem of insufficient and insufficient effective information in bioactive peptide data.

Conclusion

Accurately identifying bioactive peptides and their functional activities is crucial for the design of functional peptides and the development of therapeutic approaches. In this work, we propose a deep neural network based BPFun for predicting the functional activity of multi-functional bioactive peptides. In this model, five different feature representation methods including sequence information were used, and the data was expanded. Then, features were extracted using CNNs of different scales, and attention mechanisms were added to process information. The prediction results of the BPFun method indicate that compared with existing bioinformatics multi-label methods and deep learning methods, the BPFun method is relatively better, or at least can supplement future computing models in this field. In the future, we will continue to study the direction of multiple feature information working together, consider more efficient feature extraction and combination methods and different algorithms, and try to select representative feature information with other feature information; we will also focus on solving the problem of data imbalance to further improve the prediction performance of multifunctional bioactive peptides. Models that use multiple different features as inputs need to consider more the range of information representation after combining different features, and deep neural networks can add more layers to improve the performance of the model. We hope that BPFun can provide effective help in predicting novel multifunctional peptides or producing peptide drugs.

Supplementary Information

The online version contains supplementary material available at <https://doi.org/10.1186/s12859-025-06190-5>.

Supplementary Material 1.

Acknowledgements

The authors thank the anonymous reviewers for their valuable suggestions.

Author contributions

S.Y. and H.S. conceived the algorithm, developed the program and wrote the manuscript, L.Z. helped with manuscript editing and design. All authors reviewed the manuscript.

Funding

This research was funded by Jiangsu Students' platform for innovation and entrepreneurship training program (Grant No. 202310292023Z), Natural Science Foundation of Jiangsu Province of China (Grant No. BK20230626), partly supported by the open funds of the State Key Laboratory of Plant Environmental Resilience (Grant No. SKLPERKF2401), Supported by the Open project of State Key Laboratory of Animal Biotech Breeding (Grant No. 2024SKLAB6-1), Postgraduate Research & Practice Innovation Program of Jiangsu Province (Grant No. KYCX23_3069) and Fourth Batch of Leading Innovative Talents Introduction and Training Projects under the Longcheng Talent Plan in Changzhou City (Basic Research and Innovation) (Grant No. CQ20230086).

Availability of data and materials

Codes and data are available at <https://github.com/291357657/BPFun>. The data used in this manuscript were obtained from the following sources: AMP data: <http://fdblab.csie.ncu.edu.tw/AMPfun/index.html>. ACP data: <https://webs.iitd.edu.in/raghava/anticp2/index.html>. ADP data: <http://omicsbase.com/BioDADPep/>. AHP data: <https://biochemia.uwm.edu.pl/http://crdd.osdd.net/raghava/ahtpdb/>. AIP data: <http://kurata14.bio.kyutech.ac.jp/PreAIP/download.php>. AAP data: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0136990>. AOP data: <http://services.bioinformatics.dtu.dk/service.php?AnOxPePred-1.0>.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 20 May 2024 / Accepted: 13 June 2025

Published online: 21 July 2025

References

- Zhang S, Luo L, Sun X, Ma A. Bioactive peptides: a promising alternative to chemical preservatives for food preservation. *J Agric Food Chem*. 2021;69:12369–84.
- Sánchez A, Vázquez A. Bioactive peptides: a review. *Food Qual Saf*. 2017;1:29–46.
- Daliri EBM, Lee BH, Oh DH. Current trends and perspectives of bioactive peptides. *Critic Rev Food Sci Nutr*. 2018;58:2273–84.
- Gaspar D, Veiga AS, Castanho MARB. From antimicrobial to anticancer peptides a review. *Front Microbiol*. 2013;4:294.
- Ettayapuram Ramaprasad AS, Singh S, Gajendra PSR, Venkatesan S. AntiAngioPred: a server for prediction of anti-angiogenic peptides. *PLoS ONE*. 2015;10(9):e0136990.
- Hoskin DW, Ramamoorthy A. Studies on anticancer activities of antimicrobial peptides. *Biochim Biophys Acta*. 2008;1778:357–75.
- Cao Q, Ge C, Wang X, Harvey PJ, Zhang Z, Ma Y, et al. Designing antimicrobial peptides using deep learning and molecular dynamic simulations. *Brief Bioinform*. 2023;24:bbad058.
- Wang G, Li X, Wang Z. APD3: the antimicrobial peptide database as a tool for research and education. *Nucleic Acids Res*. 2016;44:D1087–1093.
- Rosca VE, Koskimaki EJ, Rivera GC, Pandey BN, Tamiz PA, Popel SA. Anti-angiogenic peptides for cancer therapeutics. *CPB*. 2011;12:1101–16.
- Szekanecz Z, Koch AE. Angiogenesis and its targeting in rheumatoid arthritis. *Vascul Pharmacol*. 2009;51:1–7.
- Kong J-S, Yoo S-A, Kang J-H, Ko W, Jeon S, Chae C-B, et al. Suppression of neovascularization and experimental arthritis by D-form of anti-flt-1 peptide conjugated with mini-PEG™. *Angiogenesis*. 2011;14:431–42.
- Edwards AK, Olariu I, Nakamura DS, Ahn SH, Tayade C. Chronic effects of an anti-angiogenic thrombospondin-1 mimetic peptide, ABT-898, on female mouse reproductive outcomes. *Reprod Biol Endocrinol*. 2016;14:56.
- Sarmadi BH, Ismail A. Antioxidative peptides from food proteins: a review. *Peptides*. 2010;31:1949–56.
- Lee H, Lee S, Lee I, Nam H. AMP-BERT: prediction of antimicrobial peptide function based on a BERT model. *Protein Sci*. 2023;32: e4529.
- Zhang Y, Lin J, Zhao L, Zeng X, Liu X. A novel antibacterial peptide recognition algorithm based on BERT. *Brief Bioinform*. 2021;22:bbab200.
- Pang Y, Yao L, Xu J, Wang Z, Lee T-Y. Integrating transformer and imbalanced multi-label learning to identify antimicrobial peptides and their functional activities. *Bioinformatics*. 2022;38:5368–74.
- Xu J, Li F, Li C, Guo X, Landersdorfer C, Shen H-H, et al. iAMPCN: a deep-learning approach for identifying antimicrobial peptides and their functional activities. *Brief Bioinform*. 2023;24:bbad240.
- Chen X, Zhang W, Yang X, Li C, Chen H. ACP-DA: improving the prediction of anticancer peptides using data augmentation. *Front Genet*. 2021;12: 698477.
- Bhattarai S, Kim K-S, Tayara H, Chong KT. ACP-ADA: a boosting method with data augmentation for improved prediction of anticancer peptides. *IJMS*. 2022;23:12194.
- Zahiri J, Khorsand B, Yousefi AA, Kargar M, Shirali HZR, Mahdevar G. AntAngioCOOL: computational detection of anti-angiogenic peptides. *J Transl Med*. 2019;17:1–6.
- Basith S, Manavalan B, Shin TH, Lee DY, Lee G. Evolution of machine learning algorithms in the prediction and design of anticancer peptides. *Curr Protein Pept Sci*. 2020;21:1242–50.
- Alotaibi F, Attique M, Khan Y. AntiFlamPred: an anti-inflammatory peptide predictor for drug selection strategies. *Comput Mater Continua*. 2021;69:1039–55.
- Charoenkwan P, Chiangjong W, Hasan MM, Nantasenamat C, Shoombuatong W. Review and comparative analysis of machine learning-based predictors for predicting and analyzing anti-angiogenic peptides. *Curr Med Chem*. 2022;29:849–64.
- Aranha MP, Spooner C, Demerdash O, Czejdo B, Smith JC, Mitchell JC. Prediction of peptide binding to MHC using machine learning with sequence and structure-based feature sets. *Biochim et Biophys Acta BBA General Sub*. 2020;1864:129535.

25. Tang W, Dai R, Yan W, Zhang W, Bin Y, Xia E, et al. Identifying multi-functional bioactive peptide functions using multi-label deep learning. *Brief Bioinform.* 2022;23:bbab414.
26. Gu J, Wang Z, Kuen J, Ma L, Shahroudy A, Shuai B, et al. Recent advances in convolutional neural networks. *Pattern Recogn.* 2018;77:354–77.
27. Arunkumar KE, Kalaga DV, Kumar CMS, Kawaji M, Brenza TM. Forecasting of COVID-19 using deep layer recurrent neural networks (RNNs) with gated recurrent units (GRUs) and long short-term memory (LSTM) cells. *Chaos Solitons Fractals.* 2021;146: 110861.
28. Kiperwasser E, Goldberg Y. Simple and accurate dependency parsing using bidirectional LSTM feature representations. *Trans Assoc Comput Linguist.* 2016;4:313–27.
29. Li Y, Li X, Liu Y, Yao Y, Huang G. MPMABP: a CNN and Bi-LSTM-based method for predicting multi-activities of bioactive peptides. *Pharmaceuticals.* 2022;15:707.
30. Zhang M-L, Wu L. Lift: multi-label learning with label-specific features. *IEEE Trans Pattern Anal Mach Intell.* 2015;37:107–20.
31. Wu G, Tian Y, Liu D. Cost-sensitive multi-label learning with positive and negative label pairwise correlations. *Neural Netw.* 2018;108:411–23.
32. Zou Z, Tian S, Gao X, Li Y. mlDEEPre: multi-functional enzyme function prediction with hierarchical multi-label deep learning. *Front Genet.* 2018;9:714.
33. Fürnkranz J, Hüllermeier E, Loza Mencía E, Brinker K. Multilabel classification via calibrated label ranking. *Mach Learn.* 2008;73:133–53.
34. Wu G, Zheng R, Tian Y, Liu D. Joint ranking SVM and binary relevance with robust low-rank learning for multi-label classification. *Neural Netw.* 2020;122:24–39.
35. Yang L, Wu X-Z, Jiang Y, Zhou Z-H. Multi-label learning with deep forest. 2019. <https://doi.org/10.48550/arXiv.1911.06557>.
36. Tsoumakas G, Vlahavas I. Random k-Labelsets: an ensemble method for multilabel classification. In: Kok JN, Koronacki J, de Mantaras RL, Matwin S, Mladenić D, Skowron A, editors. *Machine learning: ECML 2007*. Berlin: Springer; 2007. p. 406–17.
37. Kawashima S, Pokarowski P, Pokarowska M, Kolinski A, Katayama T, Kanehisa M. AAindex: amino acid index database, progress report 2008. *Nucl Acids Res.* 2008;36:D202–205.
38. Elnaggar A, Heinzinger M, Dallago C, Rehawi G, Wang Y, Jones L, et al. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE Trans Pattern Anal Mach Intell.* 2022;44:7112–27.
39. Lin Z, Akin H, Rao R, Hie B, Zhu Z, Lu W, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science.* 2023;379:1123–30.
40. Olsen TH, Yesiltas B, Marin FI, Pertseva M, García-Moreno PJ, Gregersen S, et al. AnOxPePred: using deep learning for the prediction of antioxidative properties of peptides. *Sci Rep.* 2020;10:21471.
41. Fu L, Niu B, Zhu Z, Wu S, Li W. CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics.* 2012;28:3150–2.
42. Shafiq M, Gu Z. Deep residual learning for image recognition: a survey. *Appl Sci.* 2022;12:8972.
43. Chandra A, Tünnermann L, Löfstedt T, Gratz R. Transformer-based deep learning for predicting protein properties in the life sciences. *Elife.* 2023;12: e82819.
44. Lindemann B, Müller T, Vietz H, Jazdi N, Weyrich M. A survey on long short-term memory networks for time series prediction. *Procedia CIRP.* 2021;99:650–5.
45. Mao S, Sejdic E. A review of recurrent neural network-based methods in computational physiology. *IEEE Trans Neural Netw Learn Syst.* 2023;34:6983–7003.
46. Raza A, Uddin J, Almuhaimeed A, Akbar S, Zou Q, Ahmad A. AIPs-SnTCN: predicting anti-inflammatory peptides using fastText and transformer encoder-based hybrid word embedding with self-normalized temporal convolutional networks. *J Chem Inf Model.* 2023;63:6537–54.
47. Ullah M, Akbar S, Raza A, Zou Q. DeepAVP-TPPred: identification of antiviral peptides using transformed image-based localized descriptors and binary tree growth algorithm. *Bioinformatics.* 2024;40:btac305.
48. Akbar S, Raza A, Zou Q. Deepstacked-AVPs: predicting antiviral peptides using tri-segment evolutionary profile and word embedding based multi-perspective features with deep stacking model. *BMC Bioinformatics.* 2024;25:102.
49. Akbar S, Zou Q, Raza A, Alarfaj FK. iAFPs-Mv-BiTCN: predicting antifungal peptides using self-attention transformer embedding and transform evolutionary based multi-view features with bidirectional temporal convolutional networks. *Artif Intell Med.* 2024;151: 102860.
50. Crooks GE, Hon G, Chandonia J-M, Brenner SE. WebLogo: a sequence logo generator. *Genome Res.* 2004;14:1188–90.
51. Becht E, McInnes L, Healy J, Dutertre C-A, Kwok IWH, Ng LG, et al. Dimensionality reduction for visualizing single-cell data using UMAP. *Nat Biotechnol.* 2018. <https://doi.org/10.1038/nbt.4314>.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.